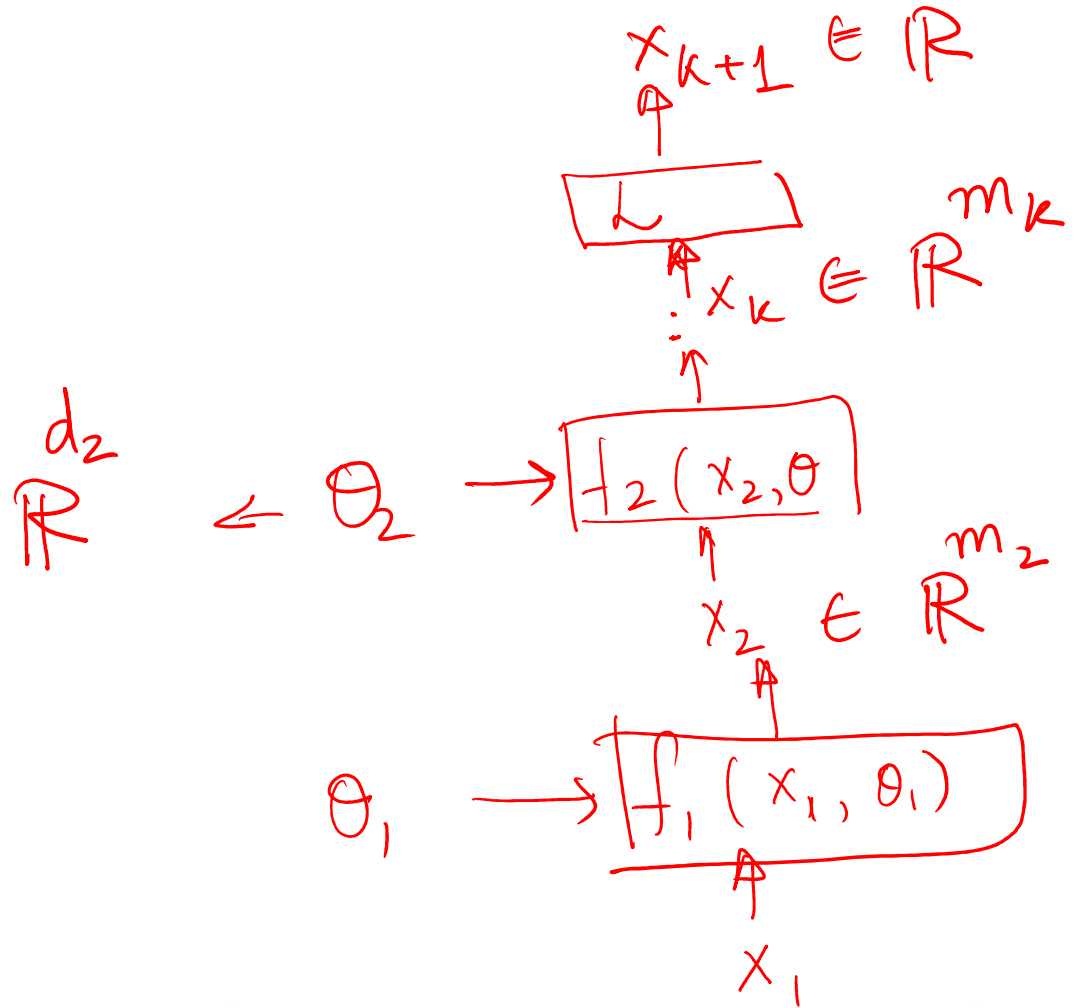
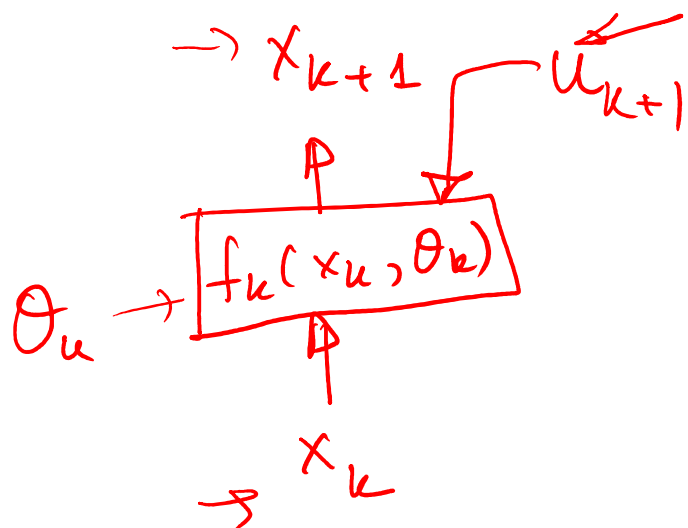

Algorithm 13.3: Backpropagation for an MLP with K layers

```
1 // Forward pass
2  $\mathbf{x}_1 := \mathbf{x}$ 
3 for  $k = 1 : K$  do
4    $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)$ 
5 // Backward pass
6  $\mathbf{u}_{K+1} := 1$ 
7 for  $k = K : 1$  do
8    $\mathbf{g}_k := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \boldsymbol{\theta}_k}$ 
9    $\mathbf{u}_k^\top := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \mathbf{x}_k}$ 
10 // Output
11 Return  $\mathcal{L} = \mathbf{x}_{K+1}$ ,  $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{u}_1$ ,  $\{\nabla_{\boldsymbol{\theta}_k} \mathcal{L} = \mathbf{g}_k : k = 1 : K\}$ 
```





$$u_{k+1}^T J_{f_k}(x_k) = u_k$$

$$u_{k+1}^T J_{f_k}(\theta_k) = \frac{\partial L}{\partial \theta_k}$$

For ease of notation, we will rewrite as:

$$z \in \mathbb{R}^m$$



Backprop vector: Given

$$u = \frac{\partial L}{\partial z}$$

$$\theta \in \mathbb{R}^d$$

$$x \in \mathbb{R}^n$$

We need to compute:

$$u^T \frac{\partial z}{\partial x} = u^T J_f(x); \quad u^T \frac{\partial z}{\partial \theta} = u^T J_f(\theta)$$

Common functions:

$$z = f(x; \theta) = \underline{W}x$$

$$W \in \mathbb{R}^{m \times n}$$

$$\theta = \bar{W}$$

$$J_f(x) = \frac{\partial z}{\partial x} = W$$

$$J_f(\theta) = \frac{\partial z}{\partial W}$$

what is the $(i, (j, k))^{\text{th}}$ entry m_i of the Jacobian

$W \rightarrow$ flatten as a vector but indexed as (j, k)

$$\begin{bmatrix} z_1 \\ \vdots \\ z_m \end{bmatrix} =$$

$$\begin{matrix} & k & n \\ \begin{matrix} \text{row} \\ j \end{matrix} & \begin{bmatrix} & & & \\ & \theta & & \\ & & & \end{bmatrix} & \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \end{matrix}$$

Real that if m
 $z = f(x) \in \mathbb{R}^m$
 $x \in \mathbb{R}^n$

$$J_f(x) = m$$

$$\left[\begin{matrix} \frac{\partial f_i}{\partial x_j} \end{matrix} \right]$$

$$j \begin{bmatrix} f_1(x) \\ \vdots \\ f_m(x) \end{bmatrix}$$

$$f(x) =$$

$$J_f(w)_{[i, j, w]} = 0 \quad \text{if } i \neq j$$

$$= x_k$$

Case 2: Consider a $f(x, \theta) =$ activation function

$$J_f(x) = \begin{bmatrix} f'(x_1) & 0 \\ 0 & f'(x_n) \end{bmatrix} \quad f\left(\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}\right) = \begin{bmatrix} g(x_1) \\ \vdots \\ g_n(x_n) \end{bmatrix} = g(x); \quad \theta = \phi$$

$$= \text{diag}([g'(x_1) \dots g'(x_n)])$$

$$u^T J_f(x) = \begin{bmatrix} u_1 \cdot g'(x_1) \\ \vdots \\ u_n \cdot g'(x_n) \end{bmatrix}^T$$

③ case : 3

$f(x)$ = Loss function : Cross Entropy (y, x)

Let # of classes be k

$$= - \sum_{c=1}^k y_c \log p_c$$

$\rightarrow \boxed{x_{\text{ent}}}$
 $\uparrow$
 $(x_1, \dots, x_k)^T$

where $p_c = \frac{e^{x_c}}{\sum_{k=1}^k e^{x_k}}$

$$J_f(x) = [p_1 - y_1, \dots, p_k - y_k]$$

First use the error between gold label and predicted (soft) labels.

[FeedForwardNeuralNetworks.ipynb - Colaboratory](#)
[\(google.com\)](#)

[Subderivative - Wikipedia](#)

Helpful videos

[Gradient descent, how neural networks learn | Chapter 2, Deep learning – YouTube](#)

[What is backpropagation really doing? | Chapter 3, Deep learning – YouTube](#)

[Backpropagation calculus | Chapter 4, Deep learning - YouTube](#)

Practical issues with training deep networks

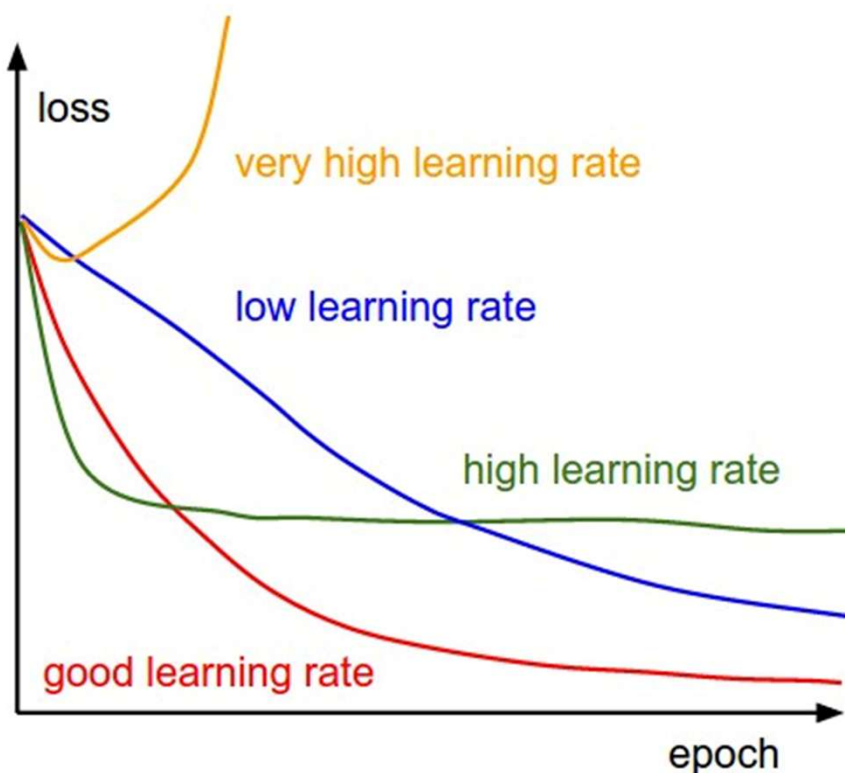
Plain SGD rarely works as-is. We will discuss number of practical challenges and some fixes next.

- Choosing learning rates
- Vanishing and exploding gradients
- Saturation of activation functions
- Residual connections
- Parameter initialization
- Parallel training

Learning Rate Schedule

Observe training losses to understand the effect of different learning rates

- Helpful to decay the learning rate over time. E.g. step decay, exponential decay, etc. (We discussed this earlier in the context of SGD)



Animation from: <http://cs231n.github.io/neural-networks-3/>

Good reference for optimizers: <https://ruder.io/optimizing-gradient-descent/>

Vanishing and Exploding gradients

- When training very deep models, the gradient tends to become either very small or very large, because the error signal is being passed through a series of layers which either amplify or diminish it.

- Explanations:

$l = \text{last layer}$

$$\frac{\partial L}{\partial \theta_{k-1}} = \frac{\partial L}{\partial x_k} \frac{\partial f_{k-1}(x_{k-1}, \theta_{k-1})}{\partial \theta_{k-1}}$$

$$\frac{\partial L}{\partial x_k} = \frac{\partial L}{\partial x_l} \frac{\partial x_l}{\partial x_{l-1}} \frac{\partial x_{l-1}}{\partial x_{l-2}} \dots \frac{\partial x_{k+1}}{\partial x_k}$$

$\nearrow l-k$ terms

Consider standard FF networks
where J_k is

$\frac{\partial x_{k+2}}{\partial x_{k+1}} \frac{\partial x_{k+1}}{\partial x_k}$

are over an activation over a linear layer

$$\frac{\partial L}{\partial x_k} = \overbrace{J_{l-1} J_{l-3} \dots J_k}^{\frac{l-k}{2} \text{ terms.}}$$

Often J_k of different layer behaves similarly

Assume $J_{l-1} = J_{l-3} = J_k = J.$

$$\frac{\partial L}{\partial x_k} = J^{(l-k)/2}$$

If $l-k$ is large then

$J^{(l-k)/2} \rightarrow \infty$ if largest eigen value (λ) of $J > 1$

$\rightarrow 0$ if $\lambda < 1$

Want

$$\lambda \approx 1$$

Fixes

- Exploding gradient: Gradient clipping

$$g_{\text{clipped}} = g \min \left(1, \frac{c}{\|g\|} \right)$$

- Vanishing gradient
 - Modify the activation functions at each layer to prevent the gradient from becoming too large or too small;
 - Modify the architecture to include residual connections so that the updates are additive rather than multiplicative
 - Modify the architecture to standardize the activations at each layer, so that the distribution of activations over the dataset remains constant during training (layer norm, batch norm etc)
 - Carefully choose the initial values of the parameters. E.g. orthogonal W so that maximum Eigen value is 1.

Saturating activation functions

- If the current input to an activation unit is that the gradient of the activation function is zero, then gradients to the lower layers is zero, and these cannot be updated.
- Saturation of various activation units

* Sigmoid activation $g(x) = \sigma(x)$
then gradient $g'(x)$ vanishes when
 x is very large or very small.

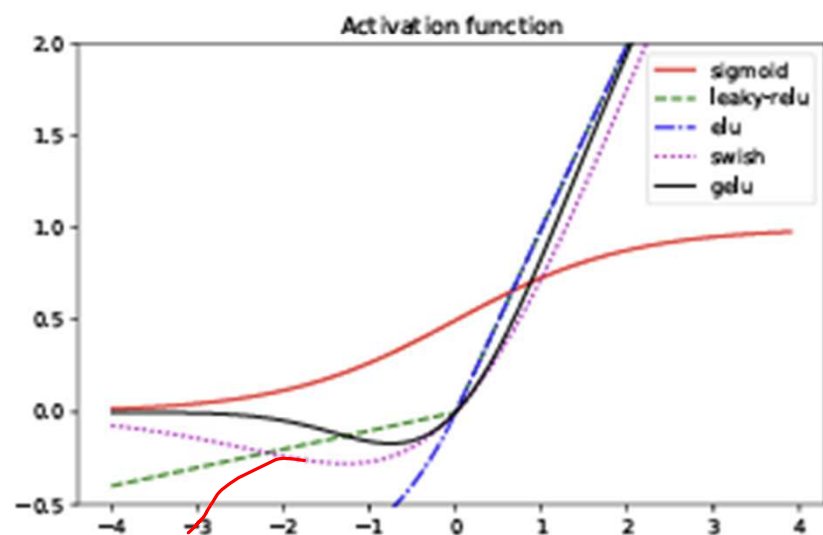
* ReLU gradient vanishes when $x \leq 0$.

* Alternatives:
Leaky ReLU $LReLU(x) = \max(\alpha x, x)$
where $\alpha < 1$

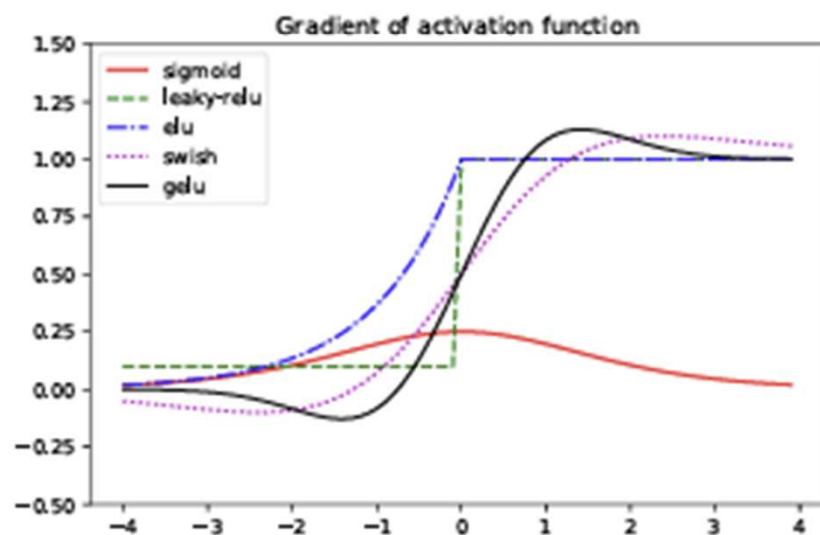
$$\begin{aligned} \text{ELU}(x, \alpha) &= \alpha(e^x - 1) && \text{if } x \leq 0 \\ &= x && \text{if } x > 0 \end{aligned}$$

Name	Definition	Range	Reference
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$[0, 1]$	
Hyperbolic tangent	$\tanh(a) = 2\sigma(2a) - 1$	$[-1, 1]$	
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$	[GBB11]
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$	[GBB11; KSH12]
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$	[MHN13]
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$	[CUH16]
Swish	$a\sigma(a)$	$[-\infty, \infty]$	[RZL17]
GELU	$a\Phi(a)$	$[-\infty, \infty]$	[HG16]

Table 13.4: List of some popular activation functions for neural networks.



(a)



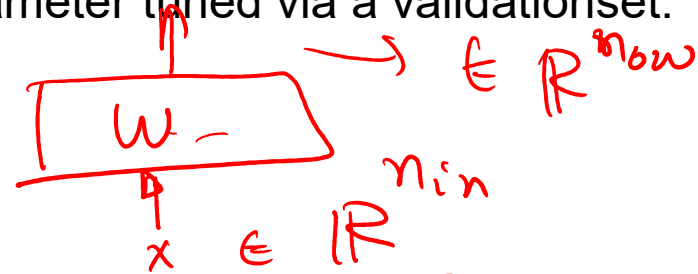
(b)

Figure 13.14: (a) Some popular activation functions. (b) Plot of their gradients. Generated by `activation_fun_deriv` [jax.ipynb](#).

Parameter Initialization

- Bad initialization could hinder convergence, cause numerical problems, saturated activation layers, slow training.
- Some guiding principles
 - Need to break symmetry for hidden units of fully connected feed-forward networks.
 - Sample from high-entropy distribution. • Typical distributions used: Uniform or Gaussian.
 - Variance (scale) of distribution important: small-scale symmetry not broken. Large scale: saturated ReLUs. Hyper-parameter tuned via a validation set. A rule of thumb is Xavier initialization

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$



- Small values for bias parameters to ensure non-saturated ~~ReLU~~ ReLUs irrespective of the input